

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# Generate a synthetic medium-to-low size image directly in memory
def generate_sample_image(width=360, height=240):
    img = np.zeros((height, width, 3), dtype=np.uint8)
    cv2.rectangle(img, (0, 0), (width, height), (200, 200, 200), -1)
    cv2.rectangle(img, (40, 40), (160, 200), (40, 40, 40), -1)
    cv2.circle(img, (260, 120), 55, (80, 80, 80), -1)
    cv2.putText(img, 'HIGH-BOOST', (45, 125), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2)
    cv2.line(img, (0, 0), (width, height), (20, 20, 20), 2)
    return img

# Load image and convert to grayscale
img_color = generate_sample_image(width=360, height=240)
gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)

# Apply Gaussian blur to get the low-frequency component
blurred = cv2.GaussianBlur(gray, (9, 9), 10.0)

# Set amplification factor A (A > 1 for high-boost filtering)
A = 1.7

# Compute high-boost filtered image: (A * Original) - Blurred
high_boost = (A * gray.astype(np.float32)) - blurred.astype(np.float32)

# Clip intensity values to valid range [0, 255] and convert to uint8
high_boost_sharpened = np.clip(high_boost, 0, 255).astype(np.uint8)

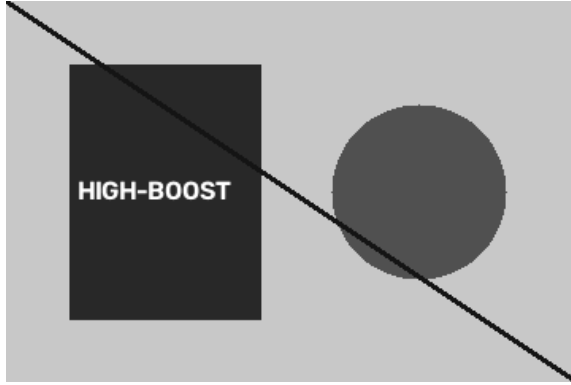
# Extract unsharp mask for visualization
unsharp_mask = cv2.convertScaleAbs(gray.astype(np.float32) - blurred.astype(np.float32))

# Display original, high-frequency mask, and sharpened output in Google Colab
print("--- 1. Original Grayscale Image (360x240) ---")
cv2_imshow(gray)

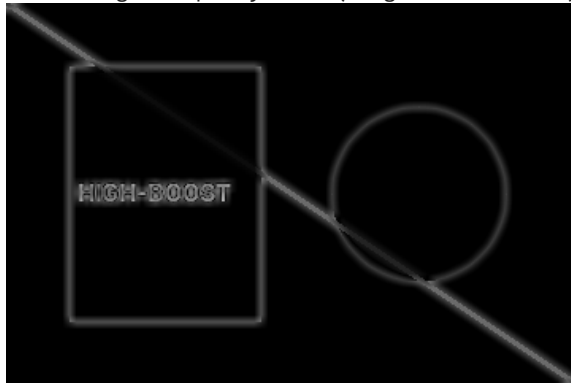
print("\n--- 2. High-Frequency Mask (Original - Blurred) ---")
cv2_imshow(unsharp_mask)

print(f"\n--- 3. High-Boost Sharpened Image (A = {A}) ---")
cv2_imshow(high_boost_sharpened)
```

--- 1. Original Grayscale Image (360x240) ---



--- 2. High-Frequency Mask (Original - Blurred) ---



--- 3. High-Boost Sharpened Image ($A = 1.7$) ---

